



```
In [3]: import os
import torch
import kagglehub
import matplotlib.pyplot as plt
from torchvision import transforms
from torchvision.datasets import OxfordIIITPet
from torch.utils.data import DataLoader

print(f"PyTorch Version: {torch.__version__}")
```

PyTorch Version: 2.4.1+cu121

```
In [4]: path = kagglehub.dataset_download("tanlikesmath/the-oxfordiiit-pet-dataset")

image_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

def to_binary_mask(mask):
    mask = transforms.functional.to_tensor(mask)
    mask = (mask * 255).long()
    binary_mask = torch.where(mask == 1, 1.0, 0.0)
    return binary_mask

dataset = OxfordIIITPet(
    root=path,
    split="trainval",
    target_types="segmentation",
    transform=image_transform,
    target_transform=to_binary_mask,
    download=True
)

train_loader = DataLoader(dataset, batch_size=16, shuffle=True)

print(f"Dataset loaded! Total images: {len(dataset)}")
```

Warning: Looks like you're using an outdated `kagglehub` version, please consider updating (latest version: 1.0.0)
Dataset loaded! Total images: 3680

```
In [5]: import torch
from torchvision import transforms
from torchvision.datasets import OxfordIIITPet
from torch.utils.data import DataLoader

image_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
```

```

def to_binary_mask(mask):
    mask = transforms.functional.resize(mask, (224, 224), interpolation=transf

    mask = transforms.functional.to_tensor(mask)
    mask = (mask * 255).long()

    binary_mask = torch.where(mask == 1, 1.0, 0.0)
    return binary_mask

dataset = OxfordIIITPet(
    root=path,
    split="trainval",
    target_types="segmentation",
    transform=image_transform,
    target_transform=to_binary_mask,
    download=True
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

train_loader = DataLoader(
    dataset,
    batch_size=16,
    shuffle=True,
    pin_memory=True if torch.cuda.is_available() else False
)

# 5. TEST: This should work now!
images, masks = next(iter(train_loader))
print(f"Batch loaded! Image shape: {images.shape}, Mask shape: {masks.shape}")

```

Batch loaded! Image shape: torch.Size([16, 3, 224, 224]), Mask shape: torch.Size([16, 1, 224, 224])

```

In [6]: images, masks = next(iter(train_loader))
fig, ax = plt.subplots(3, 2, figsize=(10, 12))
for i in range(3):
    ax[i, 0].imshow(images[i].permute(1, 2, 0))
    ax[i, 0].set_title("Original Image")
    ax[i, 0].axis("off")
    ax[i, 1].imshow(masks[i].squeeze(), cmap='gray')
    ax[i, 1].set_title("Binary Mask (Target)")
    ax[i, 1].axis("off")

plt.tight_layout()
plt.show()

```

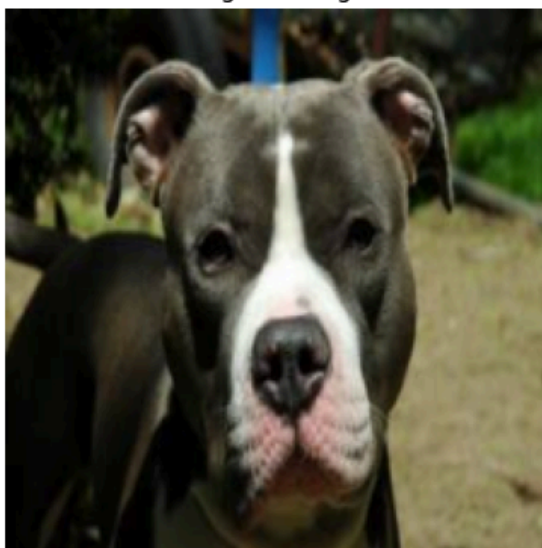
Original Image



Binary Mask (Target)



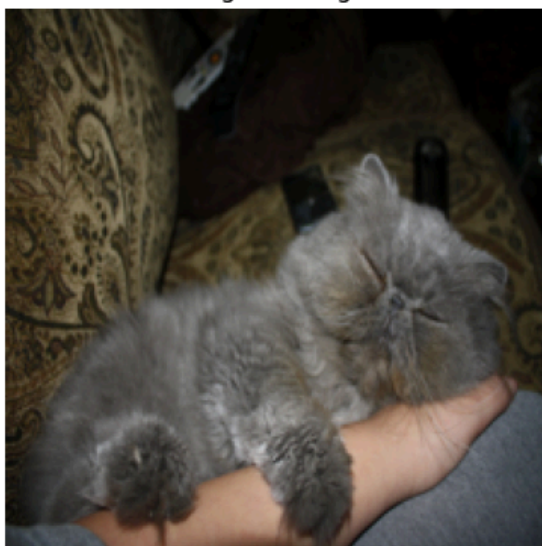
Original Image



Binary Mask (Target)



Original Image



Binary Mask (Target)



```
In [8]: import torch.nn as nn
```

```
class SimpleUNet(nn.Module):
    def __init__(self):
        super(SimpleUNet, self).__init__()

        def double_conv(in_c, out_c):
            return nn.Sequential(
                nn.Conv2d(in_c, out_c, kernel_size=3, padding=1),
                nn.BatchNorm2d(out_c),
                nn.ReLU(inplace=True),
                nn.Conv2d(out_c, out_c, kernel_size=3, padding=1),
                nn.BatchNorm2d(out_c),
                nn.ReLU(inplace=True)
            )

        self.down1 = double_conv(3, 64)
        self.pool = nn.MaxPool2d(2)
        self.down2 = double_conv(64, 128)
        self.down3 = double_conv(128, 256)

        self.up2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
        self.up_conv2 = double_conv(256, 128)
        self.up1 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        self.up_conv1 = double_conv(128, 64)

        self.final_conv = nn.Conv2d(64, 1, kernel_size=1)

    def forward(self, x):
        # Encoder
        c1 = self.down1(x)
        p1 = self.pool(c1)
        c2 = self.down2(p1)
        p2 = self.pool(c2)
        c3 = self.down3(p2)

        # Decoder with Skip Connections
        up2 = self.up2(c3)
        merge2 = torch.cat([up2, c2], dim=1)
        c4 = self.up_conv2(merge2)

        up1 = self.up1(c4)
        merge1 = torch.cat([up1, c1], dim=1)
        c5 = self.up_conv1(merge1)

        return self.final_conv(c5)

model = SimpleUNet().to(device)
print(f"Model loaded on {device}")
```

Model loaded on cuda

```
In [7]: import torch.optim as optim
```

```
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)
```

```
In [8]: train_loader = DataLoader(
        dataset,
        batch_size=2,
        shuffle=True,
        num_workers=2,
        pin_memory=True
    )
```

```
In [9]: from torch.utils.data import DataLoader, Subset

subset_indices = list(range(200))
train_subset = Subset(dataset, subset_indices)

train_loader_mini = DataLoader(
    train_subset,
    batch_size=2,
    shuffle=True,
    pin_memory=True
)

print(f"Mini-dataset ready: {len(train_subset)} images.")
```

Mini-dataset ready: 200 images.

```
In [19]: import torch.nn as nn
import torch.optim as optim
import gc

gc.collect()
torch.cuda.empty_cache()

class ThinUNet(nn.Module):
    def __init__(self):
        super(ThinUNet, self).__init__()
        def double_conv(in_c, out_c):
            return nn.Sequential(
                nn.Conv2d(in_c, out_c, kernel_size=3, padding=1),
                nn.BatchNorm2d(out_c),
                nn.ReLU(inplace=True),
                nn.Conv2d(out_c, out_c, kernel_size=3, padding=1),
                nn.BatchNorm2d(out_c),
                nn.ReLU(inplace=True)
            )
        self.down1 = double_conv(3, 32)
        self.pool = nn.MaxPool2d(2)
        self.down2 = double_conv(32, 64)
        self.down3 = double_conv(64, 128)
        self.up2 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
        self.up_conv2 = double_conv(128, 64)
```

```

        self.up1 = nn.ConvTranspose2d(64, 32, kernel_size=2, stride=2)
        self.up_conv1 = double_conv(64, 32)
        self.final_conv = nn.Conv2d(32, 1, kernel_size=1)

    def forward(self, x):
        c1 = self.down1(x); p1 = self.pool(c1)
        c2 = self.down2(p1); p2 = self.pool(c2)
        c3 = self.down3(p2)
        up2 = self.up2(c3); merge2 = torch.cat([up2, c2], dim=1); c4 = self.up
        up1 = self.up1(c4); merge1 = torch.cat([up1, c1], dim=1); c5 = self.up
        return self.final_conv(c5)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = ThinUNet().to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.BCEWithLogitsLoss()
scaler = torch.amp.GradScaler('cuda')

model.train()
print(f"Training starting on {device}...")

for epoch in range(10):
    running_loss = 0.0
    for images, masks in train_loader_mini:
        images, masks = images.to(device), masks.to(device)

        optimizer.zero_grad()

        with torch.amp.autocast('cuda'):
            outputs = model(images)
            loss = criterion(outputs, masks)
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

        running_loss += loss.item()

    avg_loss = running_loss / len(train_loader_mini)
    print(f"Epoch [{epoch+1}/10] | Loss: {avg_loss:.4f}")

print("Training Complete!")

```

```
Training starting on cuda...
Epoch [1/10] | Loss: 0.5931
Epoch [2/10] | Loss: 0.5495
Epoch [3/10] | Loss: 0.5104
Epoch [4/10] | Loss: 0.4835
Epoch [5/10] | Loss: 0.4664
Epoch [6/10] | Loss: 0.4439
Epoch [7/10] | Loss: 0.4494
Epoch [8/10] | Loss: 0.4296
Epoch [9/10] | Loss: 0.4165
Epoch [10/10] | Loss: 0.4083
Training Complete!
```

```
In [20]: import matplotlib.pyplot as plt
import torch

model.eval()
images, masks = next(iter(train_loader_mini))

with torch.no_grad():
    images = images.to(device)
    # The model gives us 'logits'
    logits = model(images)
    # Turn logits into probabilities (0.0 to 1.0)
    probs = torch.sigmoid(logits)
    # If prob > 0.5, it's a Pet (1), otherwise Background (0)
    preds = (probs > 0.5).float()

# 4. Move everything to CPU for plotting with Matplotlib
images = images.cpu()
masks = masks.cpu()
preds = preds.cpu()

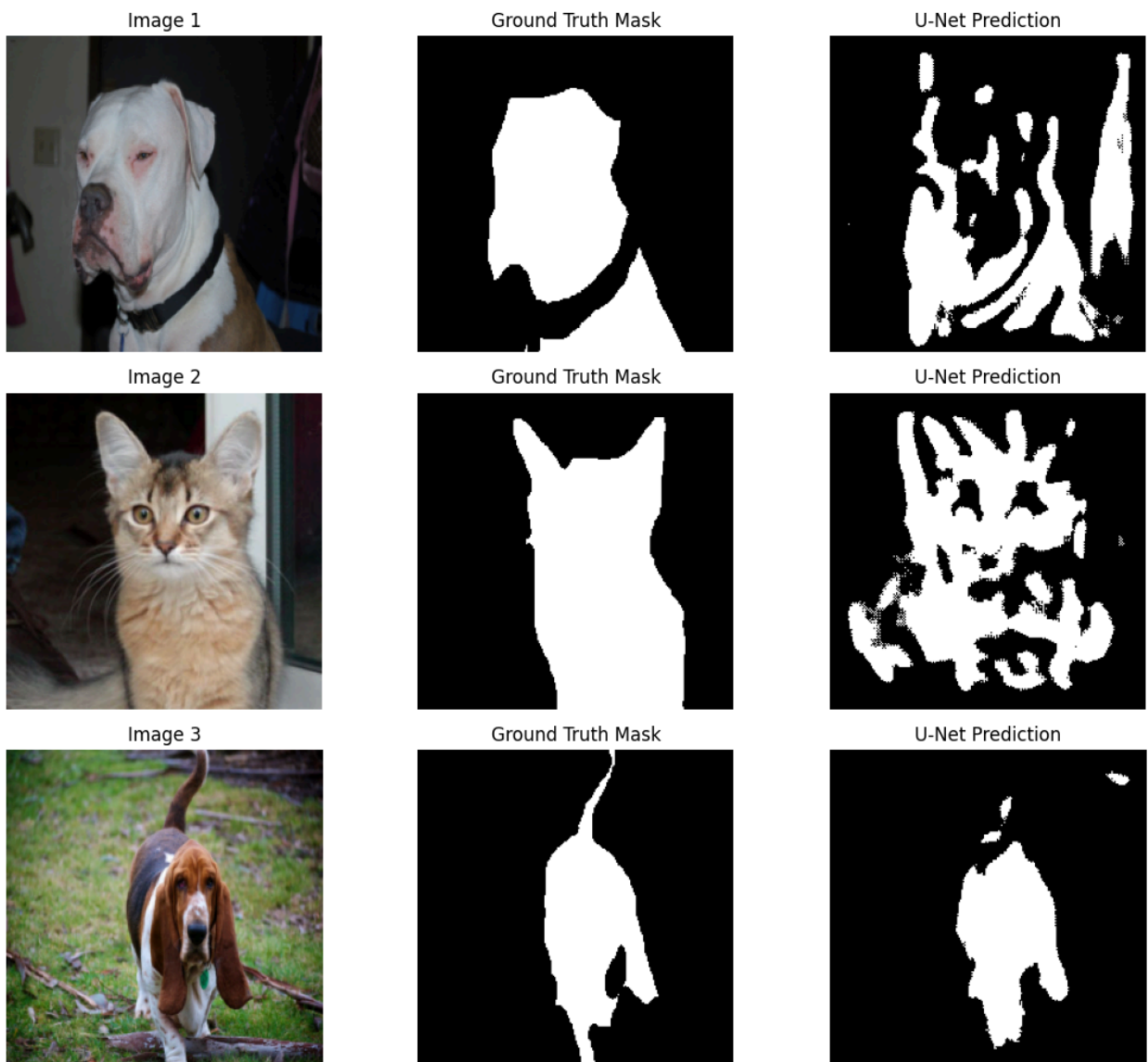
fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(12, 10))

for i in range(3):
    axes[i, 0].imshow(images[i].permute(1, 2, 0))
    axes[i, 0].set_title(f"Image {i+1}")
    axes[i, 0].axis('off')

    # Column 1: The "True" Answer (Ground Truth)
    axes[i, 1].imshow(masks[i].squeeze(), cmap='gray')
    axes[i, 1].set_title("Ground Truth Mask")
    axes[i, 1].axis('off')

    # Column 2: The Model's Best Guess
    axes[i, 2].imshow(preds[i].squeeze(), cmap='gray')
    axes[i, 2].set_title("U-Net Prediction")
    axes[i, 2].axis('off')

plt.tight_layout()
plt.show()
```



```
In [10]: import torch
import torch.nn as nn
from torchvision import models

# 1. Load the Pre-trained DeepLabV3 model
# 'weights' tells PyTorch to download the expert knowledge already learned
model = models.segmentation.deeplabv3_resnet50(weights='DeepLabV3_ResNet50_Wei

# 2. Adjust the "Head" for Binary Segmentation
# The original model was made for 21 classes. We only need 1 (Pet vs Background)
model.classifier[4] = nn.Conv2d(256, 1, kernel_size=(1, 1))

# Move to GPU
model = model.to(device)

# 3. Setup Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.BCEWithLogitsLoss()
```



```
scaler = torch.amp.GradScaler('cuda')

print("Expert model (DeepLabV3) loaded and ready!")
```

Expert model (DeepLabV3) loaded and ready!

```
In [ ]: import torch

# 1. Define the Accuracy Function
def get_accuracy(SR, GT, threshold=0.5):
    SR = torch.sigmoid(SR) > threshold
    SR = SR.float()
    GT = GT.float()
    corr = torch.sum(SR == GT)
    total_pixels = torch.numel(SR)
    return float(corr) / float(total_pixels)

# 2. Set model to training mode
model.train()

print(f"Starting Training on {device} (10 Epochs)...")
print("Please wait, calculating results for each epoch...")
print("-" * 60)

# 3. Training Loop
for epoch in range(10):
    running_loss = 0.0
    running_acc = 0.0

    for images, masks in train_loader_mini:
        images, masks = images.to(device), masks.to(device)
        optimizer.zero_grad()

        # Mixed Precision Forward Pass
        with torch.amp.autocast('cuda'):
            outputs = model(images)['out']
            loss = criterion(outputs, masks)

        # Backward Pass
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

        # Accumulate metrics
        running_loss += loss.item()
        running_acc += get_accuracy(outputs, masks)

    # Calculate Final Average for this Epoch
    epoch_loss = running_loss / len(train_loader_mini)
    epoch_acc = (running_acc / len(train_loader_mini)) * 100

    # Single print per epoch
    print(f"Epoch {epoch+1:02d}/10 | Average Loss: {epoch_loss:.4f} | Average
```

```
print("-" * 60)
print("--- ALL TRAINING FINISHED ---")
```

Starting Training on cuda (10 Epochs)...

Please wait, calculating results for each epoch...

```
-----
Epoch 01/10 | Average Loss: 0.5369 | Average Accuracy: 77.69%
Epoch 02/10 | Average Loss: 0.5507 | Average Accuracy: 77.20%
Epoch 03/10 | Average Loss: 0.5282 | Average Accuracy: 78.13%
```

In []:

In []:

In []: